

Modifications par rapport à l'énoncé initial :

- Question 27 : préciser que les entiers peuvent être nuls et que la condition sur le premier élément ne concerne que les tableaux non vides.
- Question 36 : inverser le sens du chemin recherché, de (p, q) vers une paire de caractères finaux différents, pour calculer les paires distinguables.

Commentaires généraux du jury :

- Le jury demande des preuves argumentées, en citant les propriétés et résultats antérieurs utilisés.

I – Premiers exemples

1. Donner, sans preuve, une description courte en langue française du langage reconnu par $\mathcal{A}_1$ (figure 1).

Les mots de longueur impaire sur $\{a, b\}$.

2. Donner, sans preuve, une description courte en langue française du langage reconnu par $\mathcal{A}_2$ (figure 2).

Les mots contenant un nombre impair de b .

3. Donner, sans preuve, une expression régulière du langage reconnu par $\mathcal{A}_1$.

$(a|b)((a|b)(a|b))^*$.

4. Donner, sans preuve, une expression régulière du langage reconnu par $\mathcal{A}_2$.

$a^*ba^*(ba^*ba^*)^*$.

5. Écrire en OCaml, sans justification, une instance du type `automate` correspondant à $\mathcal{A}_2$.

On identifie C à 0 et D à 1.

```
let a2 : automate =
  (2, [| (0, 1); (1, 0) |], [| false; true |])
```

II – États accessibles d'un automate

6. Écrire une fonction `numero : int -> int list -> int array` qui, à partir d'un entier n et d'une liste A d'entiers entre 0 et $n - 1$, renvoie un tableau T de taille n : $T[i] = -1$ si i est absent de A , et $T[i]$ est l'indice de l'une de ses occurrences sinon. Par exemple, `numero 5 [3;2;0]` peut renvoyer `[|2;-1;1;0;-1|]`. Le jury recommande un compteur et un parcours simple de la liste ; certaines solutions correctes étaient inutilement quadratiques.

On mémorise la position courante ; en cas de répétition, on conserve la dernière occurrence.

```
let numero n liste =
  let t = Array.make n (-1) in
  let rec aux position = function
    | [] -> ()
    | q :: suite ->
      t.(q) <- position;
      aux (position + 1) suite
  in
  aux 0 liste;
  t
```

Le coût est $O(n + |A|)$.

7. Écrire `etats_accessibles : automate -> int list` renvoyant les états accessibles dans l'ordre de leur première rencontre lors d'un parcours en profondeur depuis l'état initial, sans doublons. Donner sa complexité.

« Une attention toute particulière devait être consacrée à l'ordre exigé du résultat. » Le jury relève aussi des confusions avec le parcours en largeur et des oublis de marquage.

On marque chaque état avant d'explorer ses successeurs, d'abord par a .

```
let etats_accessibles (n, delta, f) =
  let vus = Array.make n false in
  let resultat = ref [] in
  let rec visiter q =
    if not vus.(q) then begin
      vus.(q) <- true;
      resultat := q :: !resultat;
      let succ_a, succ_b = delta.(q) in
      visiter succ_a;
      visiter succ_b
    end
  in
  visiter 0;
  let rec renverser acc = function
    | [] -> acc
    | q :: suite -> renverser (q :: acc) suite
  in
  renverser [] !resultat
```

Chaque état accessible est traité une seule fois, avec deux transitions à examiner. L'initialisation coûte $O(n)$, comme le temps total et l'espace utilisé.

8. Écrire `partie_accessible : automate -> automate` construisant la partie accessible de l'automate donné.

On pourra réemployer les questions 6 et 7.

« Beaucoup de candidats ont mal renuméroté les sommets du graphe ou les transitions. »

On renumérote dans l'ordre obtenu précédemment : la liste commence par 0, qui reste l'état initial. Les successeurs d'un état accessible sont accessibles.

```
let partie_accessible ((n, delta, f) as aut) =
  let accessibles = etats_accessibles aut in
  let num = numero n accessibles in
  let n' = List.length accessibles in
  let delta' = Array.make n' (0, 0) in
  let f' = Array.make n' false in
  let copier q =
    let succ_a, succ_b = delta.(q) in
    delta'.(num.(q)) <- (num.(succ_a), num.(succ_b));
    f'.(num.(q)) <- f.(q)
  in
  List.iter copier accessibles;
  (n', delta', f')
```

La complexité est $O(n)$.

III – Morphismes d'automates

III.A – Exemples de morphismes d'automates

9. Recopier et compléter, sans justification, le tableau par des états de $\mathcal{A}_2$ de sorte qu'il représente un morphisme $\varphi : \mathcal{A}_3 \rightarrow \mathcal{A}_2$.

q	$\varphi(q)$
E	
F	
G	

$\varphi(E) = \varphi(F) = C$ et $\varphi(G) = D$.

10. Donner, sans justification, un morphisme de $\mathcal{A}_4$ vers $\mathcal{A}_2$.

| $\varphi(H) = \varphi(I) = C$ et $\varphi(J) = \varphi(K) = D$.

11. Montrer qu'il n'existe pas de morphisme de $\mathcal{A}_1$ vers $\mathcal{A}_2$.

| La conservation de l'état initial et du caractère final impose $\varphi(A) = C$ et $\varphi(B) = D$. Mais $\varphi(\delta_{\mathcal{A}_1}(A, a)) = D \neq C = \delta_{\mathcal{A}_2}(\varphi(A), a)$, en contradiction avec (3).

12. Montrer qu'il n'existe pas de morphisme de $\mathcal{A}_5$ (figure 5) vers $\mathcal{A}_2$.

| La condition (4) impose $\varphi(L) = \varphi(N) = C$ et $\varphi(M) = D$. Or $\varphi(\delta_{\mathcal{A}_5}(N, a)) = D \neq C = \delta_{\mathcal{A}_2}(\varphi(N), a)$, ce qui contredit (3).

III.B – Propriétés des morphismes d'automates

13. Montrer que deux automates acceptent le même langage dès qu'il existe un morphisme de l'un vers l'autre.

« On doit citer les propriétés d'un morphisme utilisées, à chaque étape du raisonnement. »

| Pour un morphisme $\varphi : \mathcal{A} \rightarrow \mathcal{B}$, montrons par récurrence sur $|w|$ que

$$\forall q \in Q_{\mathcal{A}}, \quad \varphi(\delta_{\mathcal{A}}^*(q, w)) = \delta_{\mathcal{B}}^*(\varphi(q), w). \quad (*)$$

| Pour $w = \varepsilon$, les deux membres valent $\varphi(q)$. Pour $w = \sigma w'$, la définition de δ^* , l'hypothèse de récurrence et (3) donnent

$$\begin{aligned} \varphi(\delta_{\mathcal{A}}^*(q, \sigma w')) &= \varphi(\delta_{\mathcal{A}}^*(\delta_{\mathcal{A}}(q, \sigma), w')) \\ &= \delta_{\mathcal{B}}^*(\varphi(\delta_{\mathcal{A}}(q, \sigma)), w') \\ &= \delta_{\mathcal{B}}^*(\delta_{\mathcal{B}}(\varphi(q), \sigma), w') \\ &= \delta_{\mathcal{B}}^*(\varphi(q), \sigma w'). \end{aligned}$$

| Par (2) et (4), $\delta_{\mathcal{A}}^*(i_{\mathcal{A}}, w) \in F_{\mathcal{A}} \Leftrightarrow \delta_{\mathcal{B}}^*(i_{\mathcal{B}}, w) \in F_{\mathcal{B}}$. Les langages sont donc égaux.

14. Montrer qu'un morphisme φ entre deux automates de même nombre d'états est bijectif et que φ^{-1} est encore un morphisme. On dit alors que φ est un isomorphisme.

Le jury relève des preuves confuses du caractère bijectif.

| Une surjection entre ensembles finis de même cardinal est bijective. Vérifions les quatre conditions pour φ^{-1} :

- Elle est bijective, donc surjective.
- Par (2), $\varphi^{-1}(i_{\mathcal{B}}) = i_{\mathcal{A}}$.
- En appliquant (3) à $\varphi^{-1}(q)$ puis φ^{-1} à l'égalité obtenue, $\varphi^{-1}(\delta_{\mathcal{B}}(q, \sigma)) = \delta_{\mathcal{A}}(\varphi^{-1}(q), \sigma)$.
- Par (4), $q \in F_{\mathcal{B}} \Leftrightarrow \varphi^{-1}(q) \in F_{\mathcal{A}}$.

15. Montrer que la composition de deux morphismes est encore un morphisme.

| Soient $\varphi : \mathcal{A} \rightarrow \mathcal{B}$ et $\psi : \mathcal{B} \rightarrow \mathcal{C}$.

- $\psi \circ \varphi$ est surjective comme composée de surjections.
- $(\psi \circ \varphi)(i_{\mathcal{A}}) = \psi(i_{\mathcal{B}}) = i_{\mathcal{C}}$.
- $(\psi \circ \varphi)(\delta_{\mathcal{A}}(q, \sigma)) = \psi(\delta_{\mathcal{B}}(\varphi(q), \sigma)) = \delta_{\mathcal{C}}((\psi \circ \varphi)(q), \sigma)$.
- $q \in F_{\mathcal{A}} \Leftrightarrow \varphi(q) \in F_{\mathcal{B}} \Leftrightarrow (\psi \circ \varphi)(q) \in F_{\mathcal{C}}$.

III.C – Existence de morphismes entre automates accessibles

16. Montrer que (1) découle de (2), (3) et (4) lorsque les deux automates sont accessibles.

Pour établir la surjectivité, le jury attend un antécédent explicite pour chaque état.

| La preuve de (*) n'utilise que (3). Pour tout $q' \in Q_{\mathcal{B}}$, l'accessibilité donne un mot w avec $q' = \delta_{\mathcal{B}}^*(i_{\mathcal{B}}, w) = \varphi(\delta_{\mathcal{A}}^*(i_{\mathcal{A}}, w))$, par (2) et (*). Ainsi φ est surjective. L'accessibilité de $\mathcal{A}$ et (4) ne sont pas nécessaires ici.

17. Écrire `existe_morphisme : automate -> automate -> bool * morphisme` qui, pour deux automates accessibles, indique s'il existe un morphisme du premier vers le second et en renvoie un lorsqu'il existe. Sinon, le tableau renvoyé est quelconque. On pourra expliquer le principe de l'algorithme avant le code. *Le jury conseille de décrire l'algorithme et de commenter les différentes étapes du code.*

On impose l'image de l'état initial, puis celles des états rencontrés en profondeur. L'appel `visiter q q'` impose $\varphi(q) = q'$: si l'image est déjà fixée, on vérifie sa compatibilité ; sinon, on vérifie le caractère final et on propage aux deux successeurs.

```

let existe_morphisme (n, delta, f) (n', delta', f') =
  let phi = Array.make n (-1) in
  let possible = ref true in
  let rec visiter q q' =
    if !possible then begin
      if phi.(q) <> -1 then begin
        if phi.(q) <> q' then possible := false
      end else if f.(q) <> f'.(q') then
        possible := false
      else begin
        phi.(q) <- q';
        let qa, qb = delta.(q) in
        let qa', qb' = delta'.(q') in
        visiter qa qa';
        visiter qb qb'
      end
    end
  in
  visiter 0 0;
  (!possible, phi)

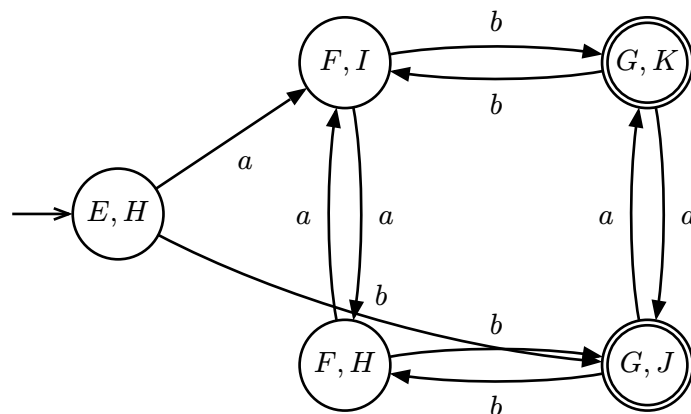
```

Chaque image est nécessaire par (2) et (3). En l'absence de contradiction, l'accessibilité donne une image à chaque état et (2), (3), (4) sont vérifiées ; la question 16 assure (1). Le caractère final de l'état initial est également testé. La complexité est $O(n)$.

IV – Constructions de morphismes d'automates

IV.A – Automate produit

18. Dessiner, sans justification, la partie accessible du produit $\mathcal{A}_3 \times \mathcal{A}_4$. *Le jury relève des incompréhensions de la définition du produit d'automates.*



19. Écrire `produit : automate -> automate -> automate` renvoyant le produit des deux automates donnés. *Le jury insiste sur la renumérotation des couples d'états et des transitions associées.*

On représente (q, q') par $q + nq'$, où n est le nombre d'états du premier automate. L'état initial $(0, 0)$ est codé par 0.

```

let produit (n, delta, f) (n', delta', f') =
  let delta_prod = Array.make (n * n') (0, 0) in
  let f_prod = Array.make (n * n') false in
  for q = 0 to n - 1 do
    for q' = 0 to n' - 1 do
      let qa, qb = delta.(q) in
      let qa', qb' = delta'.(q') in
      let k = q + n * q' in
      delta_prod.(k) <- (qa + n * qa', qb + n * qb');
      f_prod.(k) <- f.(q) && f'.(q')
    done
  done;
  (n * n', delta_prod, f_prod)

```

La complexité est $O(nn')$.

20. Soit (q, q') un état accessible du produit de deux automates acceptant le même langage. Montrer que q est final dans le premier si et seulement si q' est final dans le second.

Le jury a trouvé peu de preuves convaincantes pour cette question.

Par récurrence sur $|w|$,

$$\delta_{\mathcal{A} \times \mathcal{A}'}^*((q, q'), w) = (\delta_{\mathcal{A}}^*(q, w), \delta_{\mathcal{A}'}^*(q', w)).$$

L'accessibilité fournit donc un même mot w tel que $q = \delta_{\mathcal{A}}^*(i_{\mathcal{A}}, w)$ et $q' = \delta_{\mathcal{A}'}^*(i_{\mathcal{A}'}, w)$. Alors $q \in F_{\mathcal{A}} \Leftrightarrow w \in L(\mathcal{A}) \Leftrightarrow w \in L(\mathcal{A}') \Leftrightarrow q' \in F_{\mathcal{A}'}$.

21. Montrer qu'il existe toujours un morphisme de la partie accessible du produit de deux automates accessibles acceptant le même langage vers chacun de ces automates.

Le jury a trouvé peu de preuves convaincantes pour cette question.

Soit $\mathcal{B}$ cette partie accessible et $\varphi(q, q') = q$ la première projection.

- Pour chaque q , l'accessibilité de $\mathcal{A}$ fournit w menant à q . Avec $q' = \delta_{\mathcal{A}'}^*(i_{\mathcal{A}'}, w)$, le couple (q, q') est accessible et d'image $q : \varphi$ est surjective.
- $\varphi(i_{\mathcal{B}}) = \varphi(i_{\mathcal{A}}, i_{\mathcal{A}'}) = i_{\mathcal{A}}$.
- Par définition du produit, $\varphi(\delta_{\mathcal{B}}((q, q'), \sigma)) = \delta_{\mathcal{A}}(q, \sigma) = \delta_{\mathcal{A}}(\varphi(q, q'), \sigma)$.
- Par la question 20, $(q, q') \in F_{\mathcal{B}} \Leftrightarrow q \in F_{\mathcal{A}}$ et $q' \in F_{\mathcal{A}'} \Leftrightarrow q \in F_{\mathcal{A}}$.

La seconde projection $\psi(q, q') = q'$ est de même un morphisme vers $\mathcal{A}'$.

IV.B – Diagramme d'automates

L'alternative entre égalité des images par φ et par ψ peut changer à chaque pas de la chaîne ; le jury relève des confusions sur ce point.

22. Montrer que $\equiv$ est une relation d'équivalence sur $Q_{\mathcal{B}}$.

- Réflexivité : la suite réduite à p convient ($k = 0$).
- Symétrie : on renverse une suite reliant p à q .
- Transitivité : on concatène les suites de p à q et de q à r , en gardant une seule fois le terme commun q .

23. Montrer que $p \equiv q$ entraîne $\delta_{\mathcal{B}}(p, \sigma) \equiv \delta_{\mathcal{B}}(q, \sigma)$ pour toute lettre $\sigma \in \{a, b\}$.

Soit $p = q_0, \dots, q_k = q$ une suite témoin. Si $\varphi(q_j) = \varphi(q_{j+1})$, la condition (3) donne

$$\varphi(\delta_{\mathcal{B}}(q_j, \sigma)) = \delta_{\mathcal{A}}(\varphi(q_j), \sigma) = \delta_{\mathcal{A}}(\varphi(q_{j+1}), \sigma) = \varphi(\delta_{\mathcal{B}}(q_{j+1}, \sigma)).$$

On raisonne de même avec ψ lorsque les images par ψ sont égales. La suite $\delta_{\mathcal{B}}(q_0, \sigma), \dots, \delta_{\mathcal{B}}(q_k, \sigma)$ témoigne de l'équivalence recherchée.

24. Montrer que, si $p \equiv q$, alors p est final dans $\mathcal{B}$ si et seulement si q l'est.

Lorsque $\varphi(q_j) = \varphi(q_{j+1})$, la condition (4) donne

$$q_j \in F_B \Leftrightarrow \varphi(q_j) \in F_A \Leftrightarrow \varphi(q_{j+1}) \in F_A \Leftrightarrow q_{j+1} \in F_B.$$

On obtient la même équivalence si les images par ψ sont égales. Le caractère final est donc constant le long d'une suite témoin.

25. Construire un automate accessible $\mathcal{C}$ d'ensemble d'états $\{S_0, \dots, S_{\ell-1}\}$ tel que $\eta : \mathcal{B} \rightarrow \mathcal{C}$ soit un morphisme. Justifier.

On pose

$$Q_{\mathcal{C}} = \{S_0, \dots, S_{\ell-1}\}, \quad i_{\mathcal{C}} = [i_{\mathcal{B}}] = S_0, \\ \delta_{\mathcal{C}}([q], \sigma) = [\delta_{\mathcal{B}}(q, \sigma)], \quad F_{\mathcal{C}} = \{[q] \mid q \in F_{\mathcal{B}}\}.$$

Les questions 23 et 24 assurent que transitions et caractère final sont indépendants du représentant choisi. On obtient donc un automate déterministe complet. L'application η est surjective, préserve l'état initial et commute aux transitions par définition. La question 24 donne $q \in F_{\mathcal{B}} \Leftrightarrow [q] \in F_{\mathcal{C}}$: η est un morphisme. Enfin, l'accessibilité de $\mathcal{B}$ fournit pour chaque q un mot w menant à q . Par $(*)$, $\delta_{\mathcal{C}}^*(S_0, w) = \eta(q) = [q]$, donc $\mathcal{C}$ est accessible.

26. Construire deux morphismes $\varphi' : \mathcal{A} \rightarrow \mathcal{C}$ et $\psi' : \mathcal{A}' \rightarrow \mathcal{C}$, où $\mathcal{C}$ est l'automate de la question 25.

Pour $q \in Q_{\mathcal{A}}$, choisir p tel que $\varphi(p) = q$ et poser $\varphi'(q) = [p]$. Cet antécédent existe par surjectivité. Si $\varphi(p) = \varphi(p')$, alors $p \equiv p'$, donc $[p] = [p']$: la définition ne dépend pas du choix et $\varphi' \circ \varphi = \eta$.

- Toute classe $[p]$ est l'image de $\varphi(p)$, donc φ' est surjective.
- $\varphi'(i_{\mathcal{A}}) = \varphi'(\varphi(i_{\mathcal{B}})) = [i_{\mathcal{B}}] = i_{\mathcal{C}}$.
- Si $q = \varphi(p)$,

$$\varphi'(\delta_{\mathcal{A}}(q, \sigma)) = \varphi'(\varphi(\delta_{\mathcal{B}}(p, \sigma))) = [\delta_{\mathcal{B}}(p, \sigma)] = \delta_{\mathcal{C}}(\varphi'(q), \sigma).$$

- $q \in F_{\mathcal{A}} \Leftrightarrow p \in F_{\mathcal{B}} \Leftrightarrow [p] \in F_{\mathcal{C}} \Leftrightarrow \varphi'(q) \in F_{\mathcal{C}}$.

On définit de même $\psi'(\psi(p)) = [p]$; c'est un morphisme et $\psi' \circ \psi = \eta$.

27. Écrire `renomme` : `int array` -> `int array` qui renomme un tableau d'entiers positifs ou nuls prenant ℓ valeurs distinctes à l'aide des entiers de 0 à $\ell - 1$. Si le tableau est non vide, le premier élément du résultat doit valoir 0. Par exemple, `renomme [|4;4;5;0;4;5|]` peut renvoyer `[|0;0;1;2;0;1|]`. Préciser la complexité.

Chaque valeur reçoit un numéro lors de sa première apparition : la première reçoit donc 0.

```
let renomme t =
  let n = Array.length t in
  if n = 0 then [|]|
  else begin
    let maximum = ref t.(0) in
    for i = 1 to n - 1 do
      maximum := max !maximum t.(i)
    done;
    let code = Array.make (!maximum + 1) (-1) in
    let resultat = Array.make n 0 in
    let prochain = ref 0 in
    for i = 0 to n - 1 do
      let valeur = t.(i) in
      if code.(valeur) = -1 then begin
        code.(valeur) <- !prochain;
        incr prochain
      end;
      resultat.(i) <- code.(valeur)
    done;
    resultat
  end
```

Pour une longueur $n > 0$ et un maximum M , le temps et l'espace sont $O(n + M + 1)$. L'initialisation de code interdit de conclure à $O(n)$ pour des valeurs arbitrairement grandes. Pour le tableau vide, le coût est constant.

28. Écrire `relation : morphisme -> morphisme -> morphisme` qui, à partir de $[\varphi(q)]$ et $[\psi(q)]$, renvoie $[\eta(q)]$, c'est-à-dire un tableau t à valeurs entre 0 et $\ell - 1$ tel que $t.(p) = t.(q)$ équivaut à $p \equiv q$, et $t.(0) = 0$.

La relation décrit les composantes connexes du graphe non orienté $G = (S, A)$, où $S = Q_B$ et une arête relie p à q si $\varphi(p) = \varphi(q)$ ou $\psi(p) = \psi(q)$. On les parcourt en profondeur en commençant par 0, dont la classe reçoit le numéro 0.

```
let relation phi psi =
  let n = Array.length phi in
  let eta = Array.make n (-1) in
  let rec visiter p classe =
    if eta.(p) = -1 then begin
      eta.(p) <- classe;
      for q = 0 to n - 1 do
        if phi.(p) = phi.(q) || psi.(p) = psi.(q) then
          visiter q classe
      done
    end
  in
  let prochaine = ref 0 in
  for p = 0 to n - 1 do
    if eta.(p) = -1 then begin
      visiter p !prochaine;
      incr prochaine
    end
  done;
  eta
```

Chaque état déclenche une seule exploration de ses n voisins possibles. Le temps est $O(n^2)$ et l'espace auxiliaire $O(n)$, pile comprise.

V – Réduction d'automates

« Chacune de ces questions doit être citée dans une argumentation, au moment de leur utilisation. »

V.A – Existence et unicité

29. Montrer que, si deux automates accessibles $\mathcal{A}$ et $\mathcal{A}'$ acceptent le même langage, on peut construire un automate $\mathcal{C}$ et deux morphismes $\varphi' : \mathcal{A} \rightarrow \mathcal{C}$ et $\psi' : \mathcal{A}' \rightarrow \mathcal{C}$.

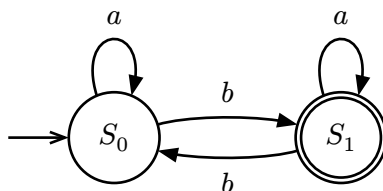
Prendre pour $\mathcal{B}$ la partie accessible de $\mathcal{A} \times \mathcal{A}'$. La question 21 fournit les projections $\varphi : \mathcal{B} \rightarrow \mathcal{A}$ et $\psi : \mathcal{B} \rightarrow \mathcal{A}'$. Les questions 25 et 26 donnent alors l'automate accessible $\mathcal{C}$ et les deux morphismes demandés.

30. Déterminer l'automate $\mathcal{C}$ de la question 29 pour $\mathcal{A}_3$ et $\mathcal{A}_4$, et préciser φ' et ψ' .

Les classes dans la partie accessible du produit sont

$$S_0 = \{(E, H), (F, H), (F, I)\}, \quad S_1 = \{(G, J), (G, K)\}.$$

En effet, $(E, H) \equiv (F, H) \equiv (F, I)$ et $(G, J) \equiv (G, K)$ par égalité d'une coordonnée. Les deux groupes ne peuvent être équivalents à cause de leur caractère final. L'automate est isomorphe à $\mathcal{A}_2$:



On a $\varphi'(E) = \varphi'(F) = S_0$, $\varphi'(G) = S_1$, $\psi'(H) = \psi'(I) = S_0$ et $\psi'(J) = \psi'(K) = S_1$.

31. Montrer que deux automates de $\mathcal{K}_L$ ayant m_L états sont nécessairement isomorphes.

La question 29 donne des morphismes $\varphi' : \mathcal{A} \rightarrow \mathcal{C}$ et $\psi' : \mathcal{A}' \rightarrow \mathcal{C}$, où $\mathcal{C}$ est accessible. Par la question 13, $\mathcal{C}$ reconnaît L , donc $|Q_{\mathcal{C}}| \geq m_L$. La surjectivité de φ' donne l'inégalité inverse. Les trois automates ont donc m_L états. Par la question 14, φ' et ψ' sont des isomorphismes, et $(\psi')^{-1} \circ \varphi'$ aussi, par la question 15.

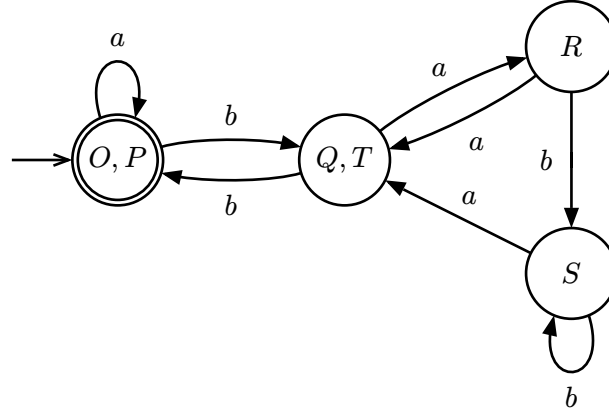
32. Montrer que, pour tout $\mathcal{A} \in \mathcal{K}_L$, il existe un morphisme $\varphi : \mathcal{A} \rightarrow \mathcal{M}_L$, où $\mathcal{M}_L$ est un automate de $\mathcal{K}_L$ à m_L états.

Comme L est régulier, il possède un automate déterministe complet ; sa partie accessible appartient à $\mathcal{K}_L$. L'ensemble non vide des nombres d'états possède un minimum m_L , atteint par un automate $\mathcal{M}_L$. Par la question 29, on obtient $\varphi' : \mathcal{A} \rightarrow \mathcal{C}$ et $\psi' : \mathcal{M}_L \rightarrow \mathcal{C}$. Comme précédemment, $\mathcal{C} \in \mathcal{K}_L$ et $m_L \leq |Q_{\mathcal{C}}| \leq m_L$. Donc ψ' est un isomorphisme et $(\psi')^{-1} \circ \varphi'$ est le morphisme demandé.

V.B – Construction d'un automate réduit par fusion d'états

33. Dessiner un automate $\mathcal{A}_6^{O,P}$ dans lequel les états O et P de $\mathcal{A}_6$ ont été fusionnés. Donner un morphisme $\mathcal{A}_6 \rightarrow \mathcal{A}_6^{O,P}$.

Fusionner O et P impose de fusionner leurs successeurs par b , soit T et Q . Ces deux fusions suffisent :



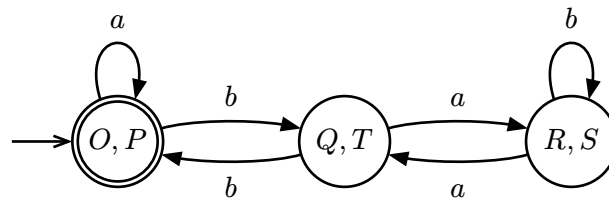
Les étiquettes désignent les ensembles d'antécédents. Le morphisme envoie O, P sur $\{O, P\}$, Q, T sur $\{Q, T\}$, R sur $\{R\}$ et S sur $\{S\}$. Il est surjectif, préserve l'état initial P et le caractère final, et commute aux deux transitions comme le montre la figure.

34. Expliquer brièvement pourquoi on ne peut construire $\mathcal{A}_6^{Q,R}$ muni d'un morphisme $\psi : \mathcal{A}_6 \rightarrow \mathcal{A}_6^{Q,R}$ tel que $\psi(Q) = \psi(R)$.

Q et R ne sont pas finaux, mais $\delta_{\mathcal{A}_6}(Q, b) = O$ est final, tandis que $\delta_{\mathcal{A}_6}(R, b) = S$ ne l'est pas. Si $\psi(Q) = \psi(R)$, (3) imposerait $\psi(O) = \delta(\psi(Q), b) = \delta(\psi(R), b) = \psi(S)$, ce que (4) interdit.

35. Quels états faut-il encore fusionner dans $\mathcal{A}_6^{O,P}$ pour obtenir un automate à trois états $\mathcal{M}_{L_6}$ reconnaissant le même langage que $\mathcal{A}_6$?

On fusionne encore R et S .



La projection de chaque état sur sa paire est un morphisme, donc conserve le langage. Les trois états sont accessibles par ε , b et ba . La classe $\{O, P\}$ se distingue des deux autres par ε ; les classes $\{Q, T\}$ et $\{R, S\}$ se distinguent par b . Aucun morphisme ne peut donc identifier deux de ces états. Le morphisme vers $\mathcal{M}_{L_6}$ fourni par la question 32 est injectif : cet automate est minimal.

36. Soit $\mathcal{A} = \langle Q, i, \delta, F \rangle$ un automate accessible. On appelle $P = (S, A)$ le graphe orienté de sommets $S = Q \times Q$, avec, pour chaque $\sigma \in \{a, b\}$, un arc de (p, q) vers $(\delta(p, \sigma), \delta(q, \sigma))$. Écrire `table_de_predecesseurs : automate -> bool array array` renvoyant une matrice $n \times n$ dont la case (p, q) vaut `true` si et seulement s'il existe un chemin de (p, q) vers un couple (p_0, q_0) dont exactement un état appartient à F . On essaiera de ne pas dépasser $O(n^2)$.

On construit les listes de prédécesseurs des sommets de P , puis on parcourt le graphe à rebours depuis les paires de caractères finaux différents.

```
let table_de_predecesseurs (n, delta, f) =
  let pred = Array.make_matrix n n [] in
  for p = 0 to n - 1 do
    for q = 0 to n - 1 do
      let pa, pb = delta.(p) in
      let qa, qb = delta.(q) in
      pred.(pa).(qa) <- (p, q) :: pred.(pa).(qa);
      pred.(pb).(qb) <- (p, q) :: pred.(pb).(qb)
    done
  done;
  let d = Array.make_matrix n n false in
  let rec visiter (p, q) =
    if not d.(p).(q) then begin
      d.(p).(q) <- true;
      List.iter visiter pred.(p).(q)
    end
  in
  for p = 0 to n - 1 do
    for q = 0 to n - 1 do
      if f.(p) <> f.(q) then visiter (p, q)
    done
  done;
  d
```

Une paire est marquée exactement lorsqu'elle peut atteindre dans P une paire contenant un seul état final. Les étiquettes du chemin forment un mot qui distingue les états ; le mot vide correspond aux paires initialement marquées. On construit $2n^2$ arcs, puis chaque sommet et chaque arc sont visités au plus une fois. Temps et espace sont donc $O(n^2)$.

Pour voir l'erreur originale, prendre deux états, 0 initial, 1 seul final, toutes les transitions allant en 1. Le parcours dans le sens original atteint $(1, 1)$ depuis $(0, 1)$ et marque donc un état comme distinguable de lui-même.

37. Décrire le principe, le justifier, puis écrire `reduit : automate -> automate` qui renvoie l'automate $\mathcal{M}_L$ associé au langage L de l'automate donné.

On commence par prendre la partie accessible, ce qui conserve le langage. Avec la table D de la question 36 corrigée, définir

$$p \sim q \Leftrightarrow D[p, q] = \text{faux} \Leftrightarrow \forall w \in \{a, b\}^*, (\delta^*(p, w) \in F \Leftrightarrow \delta^*(q, w) \in F).$$

Cette relation exprime l'égalité des langages reconnus depuis chaque état : c'est une équivalence. Le choix $w = \varepsilon$ montre que le caractère final est constant dans une classe. Si $p \sim q$, lire σw montre que $\delta(p, \sigma) \sim \delta(q, \sigma)$ pour chaque lettre σ . Comme à la question 25, on peut donc former le quotient : il est accessible et la projection est un morphisme, donc il reconnaît encore L .

Deux classes distinctes sont distinguées par un mot. Par (*) et (4), un morphisme ne peut les identifier. Le morphisme du quotient vers $\mathcal{M}_L$ fourni par la question 32 est donc injectif, et c'est un isomorphisme. Le quotient est minimal.

On parcourt les états dans l'ordre croissant pour numéroter les classes, celle de l'état initial recevant 0. Un représentant par classe suffit pour calculer les transitions.

```

let reduit aut =
  let ((n, delta, f) as accessible) = partie_accessible aut in
  let d = table_de_predecesseurs accessible in
  let classe = Array.make n (-1) in
  let representant = Array.make n 0 in
  let nombre = ref 0 in
  for p = 0 to n - 1 do
    if classe.(p) = -1 then begin
      representant.(!nombre) <- p;
      for q = p to n - 1 do
        if not d.(p).(q) then classe.(q) <- !nombre
      done;
      incr nombre
    end
  done;
  let delta' = Array.make !nombre (0, 0) in
  let f' = Array.make !nombre false in
  for k = 0 to !nombre - 1 do
    let p = representant.(k) in
    let pa, pb = delta.(p) in
    delta'.(k) <- (classe.(pa), classe.(pb));
    f'.(k) <- f.(p)
  done;
  (!nombre, delta', f')

```

Pour une entrée à N états, dont n accessibles, le temps et l'espace sont $O(N + n^2)$, donc $O(n^2)$ pour un automate accessible à n états.